

MODULE 15 · PART 4 - SEQUENCING AND TIME

Intervals and As-Of Joins

What was true at that moment, and what happens when two periods meet.

PART 4 of 8 MODULE 15 of 26 RUNS about 36 min

Why intervals are the hardest join

- ◆ **A key join either matches or it does not.** One row, or none, and the planner knows which.
- ◆ **An interval join can match zero, one, or several,** and which of those happened is a property of your data.
- ◆ **So the row count is not predictable from the schema,** which is why nothing warns you when it changes.
- ◆ **Every other join hazard in SQL is a special case of this one.** Fan-out, loss, and both at once.

THE TAKEAWAY

A foreign key tells the planner and the reader what the cardinality is. An interval join has no equivalent, so the assertion has to be written by hand.

The same join, two ways

```
1 -- a key join: cardinality is known from the schema
2 JOIN dim_product p ON p.product_key = s.product_key
3     -- one row, guaranteed, because product_key is unique
4
5 -- an interval join: cardinality is a property of the data
6 JOIN price_valid v ON v.product_key = s.product_key
7     AND s.sold_date >= v.valid_from
8     AND s.sold_date < v.valid_to
9     -- zero rows if the price history starts later
10    -- two rows if two validity periods overlap
```

9 Both of these are normal, neither is an error, and both are invisible without counting.

THE TAKEAWAY

The difference is not the syntax. It is that one has a uniqueness guarantee behind it and the other has a hope.

An interval join can **duplicate rows** and **lose rows**. Usually in the same query, and never with an error.

Two numbers, measured

```
1 -- every sales line, joined to the price that applied
2
3 sales lines in the fact table          7,069,736
4
5 closed interval    [valid_from, valid_to]  5,016,025
6 half-open interval [valid_from, valid_to)  4,975,095
7
8 -- 40,930 rows duplicated by the closed boundary
9 -- 2,094,641 rows lost by the inner join
```

- 8 Every sale on a price-change day matched two prices, because both intervals contain that day.
- 9 And 29.6% of sales matched nothing at all, because they happened before the product's first recorded price.

THE TAKEAWAY

One join, two defects, no error. Both were found by counting rows, and neither is visible in any other way.

Reading an interval predicate out loud

WRITTEN

SAID

`d >= v.valid_from AND d < v.valid_to`

the moment falls inside this period

`a.valid_from < b.valid_to AND b.valid_from < a.valid_to`

the two periods overlap at all

`b.valid_from <= a.valid_from AND a.valid_to <= b.valid_to`

A sits entirely inside B

`a.valid_to = b.valid_from`

A ends exactly where B begins, and they do not overlap

`GREATEST(a.from, b.from), LEAST(a.to, b.to)`

the part they have in common

THE TAKEAWAY

Five phrases cover almost every interval question. If you can say which one you want, the SQL is already written.

Why both happen at once

- ◆ **Duplication comes from the boundary.** If two intervals both contain the change day, a moment on that day matches both.
- ◆ **Loss comes from the join type.** An inner join drops any moment that falls in no interval, and says nothing.
- ◆ **They partly cancel.** The total looked plausible, because it was 30% low and 0.8% high at the same time.
- ◆ **Which is why the only test is a row count,** asserted against the input, every single time.

THE TAKEAWAY

The two errors move the total in opposite directions. A reconciliation that only checks the total can pass while both are present.

Half-open intervals. Include the start, exclude the end. Always, for anything continuous.

Five rows, and the answers by hand

```
1 price_valid          a sale on ...
2   A  2026-01-01 .. 2026-03-14  2025-12-31 -> no match
3   B  2026-03-14 .. 2026-06-02  2026-01-01 -> A (start included)
4                                     2026-03-13 -> A
5                                     2026-03-14 -> B (end excluded)
6                                     2026-07-01 -> no match, unless
7                                     B runs to a sentinel
```

5 The boundary day. With closed intervals this row matches both A and B, and that single row is the whole defect.

THE TAKEAWAY

Two intervals and five test dates. Every failure mode in this module is visible in this fixture, and none of them needs seven million rows.

Testing an interval join

- ◆ **A moment exactly on a boundary**, which is the single most important test and the one nobody writes.
- ◆ **A moment before the first interval**, to see whether your join keeps it or deletes it.
- ◆ **A moment after the last**, which tests the sentinel and catches a null end date.
- ◆ **And two keys**, because one key cannot catch a join that forgot to match on the key at all.

THE TAKEAWAY

Four test rows. The boundary one catches the fan-out, the first-interval one catches the loss, and together they are the two numbers this module opened with.

Closed, and half-open

```
1 -- closed: both endpoints included
2 WHERE sold_date >= valid_from AND sold_date <= valid_to
3
4 price A  valid 2026-01-01 .. 2026-03-14
5 price B  valid 2026-03-14 .. 2026-06-02
6 a sale on 2026-03-14 matches BOTH
7
8 -- half-open: start included, end excluded
9 WHERE sold_date >= valid_from AND sold_date < valid_to
10
11 a sale on 2026-03-14 matches B only
```

9 One character. It removed 40,930 duplicate rows from the capstone query and changed nothing else.

THE TAKEAWAY

Half-open intervals tile the timeline exactly: no gaps, no overlaps. Closed intervals either overlap by a day or leave a hole, and there is no third option.

Why half-open wins everywhere

- ◆ **Intervals tile exactly.** One ends where the next begins, and no moment is in two of them or in none.
- ◆ **Duration is a subtraction.** `valid_to` minus `valid_from`, with no plus-one correction anywhere.
- ◆ **It works for timestamps as well as dates**, where a closed interval needs an end of 23:59:59.999 and still loses a microsecond.
- ◆ **And the closed form breaks differently at different precisions**, which is how a query works on dates and fails on timestamps.

THE TAKEAWAY

The third point is the decisive one. A closed interval on a timestamp column has no correct end value, because whatever you pick, a finer precision exists below it.

Building validity intervals

```
1 CREATE OR REPLACE TABLE price_valid AS
2 SELECT product_key,
3        effective_from AS valid_from,
4        COALESCE(LEAD(effective_from) OVER (
5                    PARTITION BY product_key
6                    ORDER BY effective_from),
7        DATE '9999-12-31') AS valid_to,
8        price
9 FROM fact_price_change
10 WHERE store_key IS NULL;
```

7 The last interval of every product has no next change, so it runs to a sentinel rather than to null.

4 LEAD turns a table of change events into a table of periods. Module six, doing modelling work.

THE TAKEAWAY

259,246 change events become 259,246 half-open intervals that tile each product's timeline exactly. Verified: zero overlaps.

Sentinel or null

- ◆ **A sentinel end date makes every comparison work**, so the current row behaves like every other row.
- ◆ **A null end means 'still current' and fails every inequality**, so the current version is silently excluded.
- ◆ **Or write the predicate to allow it**, with `valid_to IS NULL OR moment < valid_to`. Correct, and repeated everywhere.
- ◆ **Kestrel's SCD2 uses 9999-12-31**, which is the common choice and is why the as-was join works without a null check.

THE TAKEAWAY

This is module two, in a schema decision. A null that means 'no end yet' will be dropped by every inequality, and nothing warns you.

A table of **change events** and a table of **periods** hold the same facts and behave completely differently.

Change events, or periods

QUESTION	CHANGE EVENTS, AND PERIODS
What is the price now?	ORDER BY ... LIMIT 1. Or, filter to the sentinel row.
What was it on a date?	A correlated subquery. Or, one range join.
How long did a price last?	LEAD, every time. Or, a subtraction.
Did two overlap?	Not expressible directly. Or, one predicate.
Adding a change	One insert. Or, an insert and an update.

THE TAKEAWAY

Periods win on every read and lose on every write. For a warehouse, which is read-heavy and loaded in batches, that trade is not close.

Store periods, derive events

- ◆ **Source systems produce events**, because that is what happened. A price changed, at a moment.
- ◆ **Warehouses should store periods**, because that is what gets asked about. What applied, when.
- ◆ **Converting on every read is the common failure**, and it is what the correlated subquery is doing, seven million times.
- ◆ **One LEAD, once, in the load**. Then every downstream query is an ordinary join.

THE TAKEAWAY

This is the same argument as module thirteen's spells table and module fourteen's session table. Compute the shape the questions want, once, in the model.

An analyst joins every sales line to the price that applied, using validity intervals built with LEAD and a BETWEEN predicate on the sale date.

The result has more rows than fact_sales_line. The excess is small, around 0.8%, and every duplicated sale_key appears exactly twice with two different prices. The product keys and dates are correct on both copies.

What is the cause?

- ☐ A fact_price_change contains duplicate rows for some product and date combinations.
- ☐ B BETWEEN is inclusive at both ends, so a sale on a change day falls inside two adjacent intervals.
- ☐ C The join is missing the product key, so some sales are matching another product's price history.
- ☐ D LEAD returned null for the latest interval, and a null end matched every subsequent sale.

◆ PAUSE AND DECIDE

What is the cause?

A Duplicate price rows

Worth checking and it is not this. Duplicates would affect sales on every day of the interval, not only on change days.

C Missing product key

Would produce enormous fan-out and wrong products on the copies. The scenario says the product keys are correct.

B BETWEEN is inclusive

The change day belongs to both the interval that ends there and the one that starts there. 40,930 rows on the capstone data.

D A null end date

Would drop the latest interval rather than duplicate it, because every inequality against a null is unknown. The opposite symptom.

THE SIGNAL

"Exactly twice, with two different prices." Two copies and two prices means two intervals matched, and adjacent intervals can only both match at the boundary.

Inner or outer. An inner interval join **deletes the rows that matched nothing**, quietly.

The join that keeps everything

```
1 SELECT s.sale_key, s.product_key, s.sold_date, s.net_amount,  
2        p.price,  
3        p.price IS NULL AS price_unknown  
4 FROM fact_sales_line s  
5 LEFT JOIN price_valid p  
6     ON p.product_key = s.product_key  
7     AND s.sold_date >= p.valid_from  
8     AND s.sold_date <  p.valid_to;
```

- 5 LEFT JOIN, so the grain is preserved: exactly one row per sales line, always.
- 6 And an explicit flag, so a missing price is visible rather than being a null somebody coalesces to zero.

THE TAKEAWAY

The flag is the important half. A null price that reaches a downstream COALESCE becomes a price of zero, and 2.09 million lines then have a margin of 100%.

Why moments fall outside every interval

- ◆ **Before the first one.** The most common by far, and it is what the 2.09 million rows are.
- ◆ **After the last one,** which the sentinel end date prevents, and only if you used one.
- ◆ **In a genuine gap,** where the periods do not tile. A product delisted and relisted, with nothing in between.
- ◆ **All three look identical downstream,** and they need different answers. Unknown, current, and not applicable.

THE TAKEAWAY

Three causes and one symptom. Carrying a reason rather than a null is what lets the next person tell them apart without rerunning your query.

After fixing the duplication, the same price-as-at query returns 4,975,095 rows against 7,069,736 sales lines. Margin reporting built on it reconciles internally and is 29.6% below the finance figure.

Spot checks show the missing lines are concentrated in older products and in the first weeks of the period. Every line that is present has a correct price.

What is the cause?

- ☐ A The half-open fix over-corrected, and sales on change days are now matching no interval at all.
- ☐ B Duplicate product keys in dim_product mean some sales join to a product with no price history.
- ☐ C The date cast is dropping a time component, so sales after midday fall outside the interval.
- ☐ D It is an inner join, and 2.09 million sales happened before their product's first recorded price.

◆ PAUSE AND DECIDE

What is the cause?

A Over-corrected

Half-open intervals tile exactly, so a change day falls in the interval that starts there. Nothing is lost at a boundary.

C A date cast

Would lose rows uniformly across the whole period. The losses are concentrated at the start, which points at history rather than at time-of-day.

B Duplicate product keys

Would fan out rather than drop, and the scenario says every present line has a correct price.

D An inner join, and no price yet

Price history begins partway through. Everything sold before a product's first recorded price matches no interval and is dropped.

THE SIGNAL

"Concentrated in older products and the first weeks." A loss that clusters at the beginning of a period is almost always a history that starts later than the facts do.

Overlap is **one predicate**, and almost everybody writes it with four conditions instead.

The as-was join

```
1 SELECT o.order_key, o.ordered_ts, c.postcode
2   FROM fact_order o
3  LEFT JOIN dim_customer c
4    ON c.member_durable_key = o.member_durable_key
5     AND o.ordered_ts::DATE >= c.valid_from
6     AND o.ordered_ts::DATE <= c.valid_to;
```

- 5 Join on the durable key, not the surrogate. The surrogate identifies a version; the durable key identifies the person.
- 6 Kestrel's SCD2 is closed at both ends with a 9999 sentinel, so this one is <= rather than <.

THE TAKEAWAY

200,000 customer rows for 50,000 members, four versions each. The as-was join returns the version that was current when the order was placed.

As-was and as-is

- ◆ **As-was joins on the validity period.** The address they had when they ordered, which is what a dispatch report needs.
- ◆ **As-is joins on is_current.** The address they have now, which is what a mailing list needs.
- ◆ **They give different answers for every member who moved,** and both are correct for their own question.
- ◆ **Name the column.** 'Region as at order date' and 'current region' are different measures and must not share a name.

THE TAKEAWAY

Restating history is the most common reporting dispute there is. As-is silently rewrites last year's numbers every time somebody moves region.

What as-is does to history

```
1 -- as-is: always the current version
2 JOIN dim_customer c ON c.member_durable_key = o.member_durable_key
3 AND c.is_current
4
5 -- 2025 sales by region, run in January -> region A
6 -- 2025 sales by region, run in June -> region B
7 -- because the member moved in March, and nothing
8 -- about 2025 changed
```

6 The same query, the same period, two answers. Last year's numbers moved because somebody's address did.

THE TAKEAWAY

An as-is join makes history mutable. It is sometimes exactly what is wanted, and it should never be the accidental default.

Durable keys and surrogate keys

- ◆ **The surrogate key identifies one version**, so joining a fact to it pins the fact to that version forever.
- ◆ **The durable key identifies the entity**, so joining to it lets you choose which version at query time.
- ◆ **Facts should carry both**, where the surrogate records what was true and the durable key allows as-is.
- ◆ **Carrying only the surrogate makes as-is impossible**, and carrying only the durable key makes as-was a join every time.

THE TAKEAWAY

Two keys on the fact, and the report chooses. That is a modelling decision made once that removes an argument that otherwise recurs quarterly.

When it happened and **when we knew** are different clocks, and the ledger has both.

Posted, and effective

```
1 SELECT ledger_key, store_key, amount,  
2         posted_ts,      -- when the warehouse learned it  
3         effective_date  -- when it actually happened  
4 FROM fact_ledger;  
5  
6 -- March report run on 1 April    -> total X  
7 -- March report run on 1 May     -> total X + back-dated postings  
8 -- both are correct, for different questions
```

8 Filtering on `effective_date` alone gives a March figure that keeps changing. Filtering on both gives one that does not.

THE TAKEAWAY

Kestrel back-dates roughly 14% of ledger postings, by up to 45 days. A March close run in April and again in May gives two different answers, and both are right.

Asking for either answer

```
1 -- what we now believe happened in March
2 WHERE effective_date BETWEEN DATE '2026-03-01' AND DATE '2026-03-31'
3
4 -- what we believed on 1 April
5 WHERE effective_date BETWEEN DATE '2026-03-01' AND DATE '2026-03-31'
6   AND posted_ts < TIMESTAMP '2026-04-01 00:00:00'
7
8 -- and what arrived late
9 WHERE effective_date < DATE '2026-04-01'
10  AND posted_ts      >= TIMESTAMP '2026-04-01 00:00:00'
```

6 One extra predicate reproduces any historical report exactly, which is what makes a restatement explainable.

THE TAKEAWAY

Three questions from two columns. The second one is what makes a number reproducible months later, and it is one line.

What bitemporal buys you

- ◆ **Reproducibility.** Any past report can be regenerated exactly, because you can ask what was known then.
- ◆ **Explainable restatements.** The difference between two runs is a query, not an investigation.
- ◆ **At the cost of every query carrying a second predicate,** and of everybody understanding why.
- ◆ **So it is worth it where restatement matters,** which is finance, regulation and anything audited, and rarely elsewhere.

THE TAKEAWAY

Two timelines double the thinking in every query. Pay that cost where somebody will one day ask why last quarter's number changed, and not by default.

The overlap predicate

```
1 -- do a and b overlap?
2 a.valid_from < b.valid_to AND b.valid_from < a.valid_to
3
4 -- read it as: each one starts before the other ends
5
6 -- the version people write instead
7 (a.valid_from BETWEEN b.valid_from AND b.valid_to)
8 OR (a.valid_to BETWEEN b.valid_from AND b.valid_to)
9 OR (b.valid_from BETWEEN a.valid_from AND a.valid_to)
10 OR (b.valid_to BETWEEN a.valid_from AND a.valid_to)
```

2 Two comparisons, and it is complete. Every overlap case is covered, including total containment in either direction.

THE TAKEAWAY

The long version is not wrong, and it is four times the reading. It also uses BETWEEN, which brings the closed-interval boundary problem back in.

Why the short form is complete

- ◆ **Two intervals fail to overlap in exactly two ways.** A ends before B starts, or B ends before A starts.
- ◆ **So negate both and you have overlap.** A starts before B ends, and B starts before A ends. That is the whole proof.
- ◆ **Containment is covered automatically,** because an interval inside another satisfies both conditions trivially.
- ◆ **Use strict less-than with half-open intervals,** so that touching is not overlapping. Two adjacent spells do not overlap.

THE TAKEAWAY

Thinking about it as 'not disjoint' is what makes it obvious. There are two ways to be apart and everything else is together.

The thirteen ways two intervals relate

RELATION	WRITTEN WITH HALF-OPEN INTERVALS
A before B	$a.to \leq b.from$
A meets B	$a.to = b.from$
A overlaps B	$a.from < b.from \text{ AND } b.from < a.to \text{ AND } a.to < b.to$
A during B	$b.from \leq a.from \text{ AND } a.to \leq b.to$
A starts B	$a.from = b.from \text{ AND } a.to < b.to$
A finishes B	$b.from < a.from \text{ AND } a.to = b.to$
A equals B	$a.from = b.from \text{ AND } a.to = b.to$

THE TAKEAWAY

Seven, plus the six inverses, is Allen's thirteen. Most work needs only 'any overlap', and naming the exact relation is what settles an argument about a boundary.

How much do they overlap

```
1 SELECT a.promo_key, b.promo_key,  
2        GREATEST(a.effective_from, b.effective_from) AS overlap_from,  
3        LEAST(a.effective_to, b.effective_to) AS overlap_to,  
4        LEAST(a.effective_to, b.effective_to)  
5        - GREATEST(a.effective_from, b.effective_from) AS overlap_days  
6 FROM dim_promotion a  
7 JOIN dim_promotion b  
8     ON a.promo_key < b.promo_key  
9     AND a.effective_from < b.effective_to  
10    AND b.effective_from < a.effective_to;
```

- 8 a.key < b.key, or every pair appears twice and every interval overlaps itself.
- 2 The overlap starts at the later start and ends at the earlier end. Greatest and least, not min and max.

THE TAKEAWAY

Kestrel has 6,840 promotions and 2,271,422 overlapping pairs. Roughly 355 are live on any given day, so overlapping is the normal state rather than an exception.

Overlapping intervals into coverage spans.
It is **gaps and islands**, with a running
maximum.

Merging, in full

```
1 WITH ordered AS (  
2   SELECT effective_from AS f, effective_to AS t,  
3         MAX(effective_to) OVER (ORDER BY effective_from  
4           ROWS BETWEEN UNBOUNDED PRECEDING AND 1 PRECEDING)  
5         AS max_end_so_far  
6   FROM dim_promotion),  
7 grouped AS (  
8   SELECT *, SUM(CASE WHEN max_end_so_far IS NULL  
9     OR f > max_end_so_far THEN 1 ELSE 0 END)  
10     OVER (ORDER BY f ROWS UNBOUNDED PRECEDING) AS span  
11   FROM ordered)  
12 SELECT span, MIN(f) AS span_from, MAX(t) AS span_to  
13 FROM grouped GROUP BY span;
```

4 The frame ends at 1 PRECEDING: the furthest end reached BEFORE this row. Module eight's Pareto trick.

THE TAKEAWAY

The running maximum is what makes it work. Comparing with the previous row's end fails when a long interval is followed by a short one inside it.

What merging tells you

- ◆ **Kestrel's 6,840 promotions merge into one span.** Some promotion is live on every single day of the period.
- ◆ **Which is itself the finding.** A promotional uplift measured against non-promotional days has no non-promotional days to use.
- ◆ **The naive alternative is a date spine,** generating a row per promotion per day, which is hundreds of thousands of rows for this answer.
- ◆ **Merging answers it from 6,840 rows,** in effectively no time, and it generalises to any coverage question.

THE TAKEAWAY

One span, from 6,840 intervals. The query took no measurable time and the answer is a fact about the business that nobody had stated.

Commercial ask on how many days of the year at least one promotion was running, and how many days had none. An analyst generates a date spine, joins every promotion to every day it covers, and counts distinct days.

The intermediate result is around 200,000 rows and the query is slow enough that it has been moved to a nightly job. The answer it produces is correct.

What is the better approach?

- A Merge the overlapping intervals into coverage spans with a running maximum, then measure the spans and the gaps between them.
- B Keep the date spine, which is correct, and index the join columns so that the nightly job runs faster.
- C Count the promotions and multiply by their average duration, adjusting for an estimated overlap factor.
- D Walk the promotions in date order with a recursive CTE, accumulating coverage as it goes.

◆ PAUSE AND DECIDE

What is the better approach?

- A** Merge the intervals
6,840 rows in, one span out, and the gaps come free from a LEAD over the spans. The answer is a property of the intervals.
- B** Index the spine
Makes a correct 200,000-row expansion faster. It is still expanding intervals into days to answer a question about intervals.
- C** Estimate the overlap
Replaces a correct answer with an approximation, for a question that has an exact answer computable in milliseconds.
- D** A recursive CTE
Expressible and far heavier. Merging needs one ordered pass, and recursion cannot parallelise across its passes at all.

THE SIGNAL

"The answer it produces is correct." It is, and it materialises 200,000 rows to answer a question about 6,840 intervals. Correctness was never the issue.

Containment and coverage

```
1 -- does this interval cover the whole period?
2 SELECT p.product_key,
3        BOOL_OR(v.valid_from <= DATE '2026-01-01'
4              AND v.valid_to > DATE '2026-12-31') AS single_price_all_year,
5        MIN(v.valid_from) AS first_priced,
6        COUNT(*)          AS price_periods
7 FROM dim_product p
8 LEFT JOIN price_valid v ON v.product_key = p.product_key
9 GROUP BY p.product_key;
```

- 4 One interval containing the whole year, which is different from several intervals that together cover it.

THE TAKEAWAY

Containment asks whether one period holds another. Coverage asks whether several together do, and that needs merging first.

Gaps in coverage

- ◆ **A gap means no period was in force**, which for a price means the product had no price at all that day.
- ◆ **Merge first, then LEAD across the merged spans.** The gaps are what sits between them, from module thirteen.
- ◆ **And anchor to the reporting period**, or a gap before the first span and after the last one is invisible.
- ◆ **Kestrel's 2.09 million unpriced sales are exactly this**, at the leading edge, and the anchor is what reveals them.

THE TAKEAWAY

The unpriced sales were a coverage gap before the first interval. Module thirteen said absence at the edges is invisible unless you supply the edges, and this is that.

Sometimes a moment **really does** fall in several periods, and collapsing it is the defect.

Which promotion applied

```
1 SELECT s.sale_key, p.promo_key, p.promo_name
2   FROM fact_sales_line s
3   JOIN dim_promotion p
4     ON s.sold_ts::DATE >= p.effective_from
5     AND s.sold_ts::DATE < p.effective_to
6  QUALIFY ROW_NUMBER() OVER (
7      PARTITION BY s.sale_key
8      ORDER BY p.effective_from DESC, p.promo_key) = 1;
```

8 Most recently started wins, with a tiebreaker. That is a rule somebody chose, and it should be written down.

THE TAKEAWAY

The QUALIFY is a business rule wearing a deduplication's clothes. Most recent, deepest discount, or highest priority are all defensible and give different answers.

Three answers to several matches

- ◆ **Pick one, by a stated rule.** The grain is preserved and the other matches are discarded, deliberately.
- ◆ **Keep them all, and change the grain.** One row per sale per promotion, which is honest and fans out.
- ◆ **Split the value between them,** so each promotion gets a share. The grain is preserved and the measure is allocated.
- ◆ **All three are correct, and they sum differently.** Only the first and third preserve the total.

THE TAKEAWAY

Keeping them all is the one that breaks reconciliation, because summing `net_amount` across the fanned-out rows counts every multi-promotion sale several times.

Splitting the value

```
1 WITH matched AS (  
2     SELECT s.sale_key, s.net_amount, p.promo_key,  
3           COUNT(*) OVER (PARTITION BY s.sale_key) AS n_promos  
4     FROM fact_sales_line s  
5     JOIN dim_promotion p  
6         ON s.sold_ts::DATE >= p.effective_from  
7         AND s.sold_ts::DATE < p.effective_to)  
8     SELECT promo_key,  
9           SUM(net_amount / n_promos) AS allocated_amount  
10    FROM matched GROUP BY promo_key;
```

3 Count the matches per fact, on the fanned-out rows, then divide by it. The total is preserved exactly.

THE TAKEAWAY

Equal allocation is the simplest rule and rarely the right one. Weighting by discount depth or by promotion priority is the same query with a different denominator.

Allocation, and when not to

- ◆ **Allocate when the total must be preserved**, which is any measure finance will reconcile.
- ◆ **Do not allocate a count.** Half a transaction is not a thing, and summing halves gives a number nobody can act on.
- ◆ **Report both.** Allocated revenue for the total, and unallocated reach for how many sales each promotion touched.
- ◆ **And state the rule.** Equal split, weighted, or first match, because the three give different league tables.

THE TAKEAWAY

Allocated revenue sums to the truth and reach does not, and both are useful. Publishing only one of them is what starts the argument about whose promotion worked.

The coverage assertion

```
1 SELECT
2   (SELECT COUNT(*) FROM fact_sales_line)           AS lines_in,
3   (SELECT COUNT(*) FROM priced)                   AS lines_out,
4   (SELECT COUNT(*) FROM priced WHERE price IS NULL) AS unpriced,
5   (SELECT COUNT(*) FROM price_valid a JOIN price_valid b
6     ON a.product_key = b.product_key
7     AND a.valid_from < b.valid_from
8     AND a.valid_to   > b.valid_from)              AS overlaps;
```

- 3 These two must be equal. That single comparison catches both fan-out and loss.
- 8 And this must be zero, checked on the interval table rather than on the join result.

THE TAKEAWAY

Four numbers. Two must be equal, one must be zero, and one is a figure you should be able to explain. That is a complete test of an interval join.

The most common interval question of all:
what was true at that moment.

Four ways to write an as-of join

```
1 -- 1. validity intervals, then a half-open range join
2 JOIN price_valid p ON p.product_key = s.product_key
3   AND s.sold_date >= p.valid_from AND s.sold_date < p.valid_to
4
5 -- 2. correlated subquery: the latest change at or before
6 (SELECT price FROM fact_price_change p
7   WHERE p.product_key = s.product_key AND p.effective_from <= s.sold_date
8   ORDER BY p.effective_from DESC LIMIT 1)
9
10 -- 3. join all earlier, then QUALIFY the latest
11 QUALIFY ROW_NUMBER() OVER (PARTITION BY s.sale_key
12   ORDER BY p.effective_from DESC) = 1
13
14 -- 4. ASOF JOIN, where the engine has it
```

14 One keyword, and three of the engines in this course do not have it.

THE TAKEAWAY

Measured on 7.07m sales lines in DuckDB: the range join 0.5s, ASOF JOIN 2.3s, the correlated subquery 2.3s. The portable form was the fastest.

Why the range join wins here

- ◆ **It does the work once.** Building the intervals is one pass with a LEAD, and then it is an ordinary join.
- ◆ **The other forms re-derive 'the latest before' for every row,** over and over, for seven million rows.
- ◆ **And the intervals are reusable.** Every question about prices over time then reads the same table.
- ◆ **But it needs the intervals to tile,** so overlapping validity rows would fan out rather than resolve.

THE TAKEAWAY

This is the modelling answer rather than the query answer. A table of periods is a better artefact than a table of change events, and everything downstream is simpler.

ASOF JOIN, by engine

ENGINE	SYNTAX
DuckDB	ASOF JOIN ... ON key = key AND ts >= ts
Snowflake	ASOF JOIN ... MATCH_CONDITION (ts >= ts) ON key = key
ClickHouse, Apache Doris	ASOF JOIN, with their own conventions.
Redshift, PostgreSQL, BigQuery	No. Build the intervals, or correlate.
SQL Server	No. APPLY with a TOP 1 is the usual form.

THE TAKEAWAY

Two engines, two different syntaxes for one feature. Snowflake separates the inequality into MATCH_CONDITION; DuckDB puts it in the ON clause.

A moment can fall in **two periods or none**.
A key never can, and that is the entire
difference.

Postgres has a type for this

- ◆ **daterange, tsrange, int4range**, and they are half-open by default, written as a closing bracket.
- ◆ The **&& operator is overlap**, and **@>** is containment. The predicates become one symbol each.
- ◆ A **GiST index makes interval joins fast**, which an ordinary B-tree on two columns cannot do well.
- ◆ And an **EXCLUDE constraint forbids overlaps at the schema level**, which is the guarantee this module has been asserting by hand.

THE TAKEAWAY

Module twelve said no constraint can express acyclicity. This is the case where a constraint can express the thing: non-overlapping intervals per key, enforced on write.

The constraint that prevents the defect

```
1 CREATE TABLE price_valid (  
2   product_key  INTEGER NOT NULL,  
3   validity     DATERANGE NOT NULL,  
4   price        NUMERIC  NOT NULL,  
5   EXCLUDE USING GIST (product_key WITH =, validity WITH &&)  
6 );  
7  
8 -- an INSERT that would overlap an existing row for that  
9 -- product is rejected. The fan-out cannot happen.
```

- 5 Equal product key, overlapping validity: forbidden. The scenario-one defect becomes impossible rather than detectable.

THE TAKEAWAY

Module twelve's lesson was that some properties cannot be constrained. This is the counter-example, and it is the strongest argument for range types there is.

Making an interval join fast

- ◆ **An inequality join cannot use an ordinary equality hash join**, so the planner reaches for a nested loop or a sort-merge.
- ◆ **Always include the equality**. Joining on `product_key` as well as the dates lets the engine hash first and compare within.
- ◆ **Keep the interval table small**. 259,246 periods joining to 7 million facts is a good shape.
- ◆ **And on Postgres, a GiST index on a range type** is the one case where the database genuinely indexes the interval itself.

THE TAKEAWAY

The equality predicate is what makes an interval join tractable. Without one, every fact has to be compared against every interval, and that is a cross join with a filter.

The equality that saves it

```
1 -- without an equality: every fact against every interval
2 JOIN price_valid v
3   ON s.sold_date >= v.valid_from AND s.sold_date < v.valid_to
4
5 -- with it: hash on the key, then compare dates within the bucket
6 JOIN price_valid v
7   ON v.product_key = s.product_key
8   AND s.sold_date >= v.valid_from AND s.sold_date < v.valid_to
```

3 7 million facts against 259,246 intervals. That is 1.8 trillion comparisons, and it is also wrong.

THE TAKEAWAY

Forgetting the key predicate is both a correctness bug and a performance one. The query returns every product's price for every sale, and takes a very long time to do it.

A team on Redshift needs the price that applied to each of 7 million sales lines. An engineer proposes ASOF JOIN, having used it on DuckDB locally where it worked immediately.

Redshift rejects the syntax. The price table holds change events with an effective_from and no end date. The same measure is needed by four other reports.

What should they do?

- A Use a correlated subquery taking the latest change at or before each sale, which is the standard portable answer.
- B Move the price-as-at workload to Snowflake, which has ASOF JOIN with MATCH_CONDITION syntax.
- C Build a validity-interval table with LEAD, then join it half-open. Portable, reusable, and the fastest of the three here.
- D Join every earlier price change and use QUALIFY to keep the latest, which Redshift supports.

◆ PAUSE AND DECIDE

What should they do?

A Correlated subquery

Portable and correct, and it re-derives the answer per row. It was four times slower than the interval join on this data.

C Build the intervals

One LEAD pass produces a table four other reports can read, and the join is then ordinary SQL that runs anywhere.

B Move to Snowflake

A platform decision to obtain one keyword. The syntax differs from DuckDB's anyway, so the local code would still need rewriting.

D Join all, then QUALIFY

Works, and it joins every earlier price change to every sale before filtering. The intermediate result is the problem.

THE SIGNAL

"The same measure is needed by four other reports." That sentence turns a query question into a modelling one, and the interval table is the artefact all five share.

Assertions for any interval join

- ◆ **Row count equals the input.** One row per fact, always. This catches both fan-out and loss in one number.
- ◆ **No two intervals for one key overlap,** checked on the interval table itself rather than on the join.
- ◆ **The unmatched count is known and explained,** rather than being whatever the inner join happened to drop.
- ◆ **And no matched interval starts after the moment,** which catches an inequality written the wrong way round.

THE TAKEAWAY

The first one alone would have caught both defects this module opened with, and it is a single comparison against a number you already know.

Counting concurrent intervals

```
1 WITH events AS (  
2   SELECT effective_from AS d, 1 AS delta FROM dim_promotion  
3   UNION ALL  
4   SELECT effective_to + 1, -1 FROM dim_promotion)  
5 SELECT d,  
6        SUM(SUM(delta)) OVER (ORDER BY d  
7                               ROWS UNBOUNDED PRECEDING) AS live_promotions  
8 FROM events  
9 GROUP BY d  
10 ORDER BY d;
```

- 4 Plus one when an interval starts, minus one the day after it ends. The running total is the concurrency.
- 6 SUM of SUM, over a grouped result. Module one's doubled aggregate, doing real work.

THE TAKEAWAY

The sweep line. Peak concurrency on Kestrel's promotions is 356, and the running total never reaches zero, which is why merging gave one span.

Five questions before an interval join

- ◆ **Do I have periods or events?** If events, build the periods first, with LEAD and a sentinel.
- ◆ **Should one moment match one period, or several?** Promotions are several by design and prices are not.
- ◆ **What should happen to a fact that matches nothing?** Decide, rather than letting the join type decide.
- ◆ **Is there an equality predicate as well?** Without one the join is a cross product and usually wrong.
- ◆ **And does the row count come out equal?** One comparison, and it catches the two failures this module is about.

THE TAKEAWAY

Five questions, and the second one is the only one that is genuinely about the business. The other four have the same answer almost every time.

What the sweep line gives you

- ◆ **Peak concurrency**, which merging cannot tell you. 356 promotions live at once on Kestrel's data.
- ◆ **The moment it peaked**, which is the row where the running total is highest.
- ◆ **Zero points are gaps**, so it answers the coverage question as a by-product.
- ◆ **And it is two passes over the intervals**, not an expansion into days. 6,840 rows become 13,680 events.

THE TAKEAWAY

Merging tells you when something was in force. The sweep line tells you how much. They are different questions and the second one is harder to ask any other way.

What Part 4 gave you

- ◆ **Module 13: runs.** Trading spells, and the flag-and-running-sum that produces any island.
- ◆ **Module 14: sessions.** The same technique in time, plus the difference between presence and sequence.
- ◆ **Module 15: intervals.** What those spells and sessions become once they have to meet other periods.
- ◆ **Five of the capstone's ten deliverables are in this part.** More than any other, and they build on each other.

THE TAKEAWAY

One technique, three applications, and the third one is where the results of the first two get used. Part 4 is the most connected part of the course.

Writing an interval join, in four steps

01

Periods

Do you have intervals, or change events? If events, build the intervals with LEAD first.

02

Boundary

Half-open, always. Include the start, exclude the end, and use a sentinel rather than a null.

03

Join type

LEFT, unless you can state why a fact with no matching period should disappear.

04

Assert

Row count in equals row count out. One comparison, and it catches both failure modes.

THE TAKEAWAY

Four steps, and steps two and three are each one word. Those two words are the difference between 4,975,095 rows and 7,069,736.

The confusions, settled

SYMPTOM	THE CAUSE
Slightly more rows than the fact table	Closed intervals. A moment on a boundary matched two periods.
Substantially fewer rows than the fact table	An inner join, and facts that predate the first period.
The current version is always missing	A null end date. Every inequality against it is unknown.
Margin is 100% on some rows	A null price coalesced to zero downstream.
Every interval overlaps itself	No $a.key < b.key$ in the self-join.
Merging splits a span it should not	Compared with the previous end, not the running maximum.

THE TAKEAWAY

The first two are the same query with one character different, and they move the total in opposite directions. A reconciliation on the total alone can miss both.

Five habits

- ◆ **Half-open intervals, everywhere.** Include the start, exclude the end, and never write BETWEEN on a period.
- ◆ **Sentinel end dates, not nulls,** so the current row behaves like every other row in every comparison.
- ◆ **LEFT JOIN by default,** with an explicit flag for the unmatched, so absence is visible rather than deleted.
- ◆ **Assert the row count against the input,** because it is one comparison and it catches both failure modes.
- ◆ **Build the interval table once,** rather than deriving 'the latest before' in every report that needs it.

THE TAKEAWAY

The first and third are one word each and would have prevented every defect in this module. The fourth is what tells you whether you remembered them.

Part 5 stops asking when things happened and starts changing the shape of the answer.